

Parcial Diseño de Software

Juan Camilo Betancourt
Santiago Figueroa
Juan José Guerrero
Saraí Mejía

Fundamentos de diseño de software

4 de septiembre de 2026

Parte 1

1. En la arquitectura de memoria de la JVM, ¿dónde se ubican respectivamente la variable de referencia flota y las instancias reales creadas tras ejecutar Vehiculo[] flota = new

Vehiculo[3]; flota[0] = new CamionDiesel();?

☐ a) Ambos elementos se almacenan de forma contigua en el Stack.

☒ b) La referencia flota vive en el Stack, mientras que el arreglo contenedor y el objeto CamionDiesel se alojan en el Heap.

☐ c) La referencia flota reside en el Heap y los objetos individuales se crean en el Stack.

☐ d) Toda la estructura se almacena en el área estática de métodos de la clase.

2. ¿Cuál es el propósito del principio de Encapsulamiento en la Programación Orientada a Objetos?

☐ a) Permitir que cualquier clase del proyecto modifique directamente los atributos de un objeto

sin pasar por métodos intermedios.

☐ b) Evitar el uso de constructores y forzar la inicialización mediante variables globales estáticas.

☒ c) Proteger el estado interno del objeto declarando atributos privados y exponiendo métodos

públicos con reglas de validación.

☐ d) Garantizar que todas las clases tengan exactamente la misma jerarquía de herencia múltiple

3. Un sistema de facturación contiene una clase Facturador que calcula impuestos, genera PDFs y se conecta a la base de datos MySQL. ¿Qué principio S.O.L.I.D. se está violando principalmente?

☐ a) Principio de Sustitución de Liskov (LSP).

☐ b) Principio de Segregación de Interfaces (ISP).

☒ c) Principio de Responsabilidad Única (SRP).

☐ d) Principio de Inversión de Dependencias (DIP).

4. ¿Qué diferencia estructural y de ciclo de vida existe entre la Agregación (◇) y la Composición (◆) en UML?

☐ a) En la Agregación la parte no puede existir sin el todo; en la Composición la parte tiene ciclo de vida independiente.

☒ b) En la Composición el objeto contenedor crea y destruye internamente sus partes; en la Agregación las partes se inyectan desde el exterior y sobreviven al contenedor.

☐ c) La Agregación se implementa con extends y la Composición con implements.

☐ d) No existe diferencia técnica; son sinónimos en el estándar UML.

5. Dada la declaración `IMetodoPago pago = new PagoPSE("Bancolombia");`
`pago.procesarTransaccion(50000);` ¿cómo se denomina el mecanismo que permite a la JVM ejecutar la versión concreta de `procesarTransaccion` en tiempo de ejecución?

☐ a) Ligadura temprana (Static Binding).

☐ b) Conversión explícita forzada (Downcasting).

☒ c) Ligadura tardía o despacho dinámico de métodos (Dynamic Binding / Polymorphism).

☐ d) Encapsulamiento estático.

Parte 2

1. Enunciado 6: Implementación de Interfaces y Herencia de Clases

Enunciado: En Java, una clase concreta puede implementar múltiples interfaces simultáneamente mediante la palabra reservada `implements`, pero solo puede heredar directamente de una superclase padre mediante `extends`.

Respuesta: Verdadero (V)

Justificación Técnica:

En el diseño del modelo de Java, se evita deliberadamente la complejidad y las ambigüedades que surgen de la herencia múltiple de clases (por ejemplo, el famoso "problema del diamante"). Por lo tanto, el lenguaje limita la herencia de clases a una sola superclase directa mediante el uso de la palabra reservada `extends`. Sin embargo, Java posibilita que una clase concreta implemente varias interfaces al mismo tiempo, separándolas con comas después de la cláusula `implements` (Joyanes Aguilar & Zahonero Martínez, 2014), para hacer posible el polimorfismo y la definición de contratos de comportamiento compartido sin heredar estado conflictivo.

2. Enunciado 7: Definición de Constructores y Tipos de Retorno

Enunciado: Si se declara un método constructor como `public void Estudiante(String nombre)`, Java lo reconoce como el constructor principal de la clase y lo invoca al escribir `new Estudiante("Ana")`.

Respuesta: Falso (F)

Justificación Técnica:

En la definición semántica y sintáctica del lenguaje Java, los constructores no deben declarar nunca un tipo de retorno, ni el vacío o void. Cuando se antepone la palabra "void" a un identificador que es el mismo que el nombre de la clase, el compilador deja de verlo como un constructor y lo convierte en un método regular de instancia. Por lo tanto, al tratar de instanciar el objeto con la instrucción `new Estudiante("Ana")`, se producirá un error de compilación porque el compilador no encontrará un constructor que contenga ese parámetro (Deitel & Deitel, 2017).

3. Enunciado 8: Modificadores de Acceso, Estáticos y Constantes

Enunciado: Una variable declarada con la combinación `public static final double TASA = 0.19`; es una constante inmutable de clase que no requiere que se cree una instancia del objeto para ser consultada.

Respuesta: Verdadero (V)

Justificación Técnica:

La combinación de modificadores mencionada tiene tres objetivos. en Java, que son específicos y estructurados. El modificador público proporciona visibilidad universal. facilitando que la variable sea accedida desde cualquier otro paquete o clase. El modificador static conecta la variable directamente con la clase en lugar de un objeto o instancia específica, lo que posibilita su invocación o consulta a través de la notación de punto desde el nombre mismo de la clase (por ejemplo, `NombreClase.TASA`) sin tener que instanciarla usando `new`. Por último, el modificador final asegura la inmutabilidad al garantizar que, una vez asignado un valor inicial, este no pueda cambiarse (Sierra & Bates, 2005).

Parte 3

9.1. Diagnóstico de Ejecución (8 pts): ¿Qué ocurre exactamente al ejecutar la Línea 10 y 11? Identifique el nombre de la excepción lanzada por la JVM y el valor de i en el momento de la falla.

R:/ En java las iteraciones se cuentan desde 0 por lo que en la línea 11 se están contando 5 iteraciones (0,1,2,3,4) en lugar de 4 y da error ya que calificaciones solo cuenta con 4 posiciones. Por lo que debemos cambiar la línea 11 de `calificaciones.length` a `calificaciones.length-1`

9.2. Análisis Numérico (6 pts): Suponiendo que el bucle se corrigiera para iterar solo hasta `calificaciones.length - 1` con una suma de 330, ¿por qué la Línea 14 guardaría 82.0 en promedio en lugar de 82.5?

R:/ En java cuando se hace una división entre 2 números enteros, por más que la variable en la que se guarda es un double el resultado es un int. Por lo que debemos cambiar a double uno de los operandos así `(double) acumulador / 4`

9.3. Refactorización (6 pts): Escriba el bloque de código corregido (bucle y cálculo) aplicando buenas prácticas de tipado.

R:/

```
sari@sari-HP-EliteBook-745-G3:~/Escritorio/Parcial_corte1$ javac ProcesadorRendimiento.java
sari@sari-HP-EliteBook-745-G3:~/Escritorio/Parcial_corte1$ java ProcesadorRendimiento.java
Promedio calculado: 82.5
```

```

1  public class ProcesadorRendimiento {
    Run | Debug
2      public static void main(String[] args) {
3          int[] calificaciones = new int[4];
4          calificaciones[0]= 80;
5          calificaciones[1]= 90;
6          calificaciones[2]= 75;
7          calificaciones[3]= 85;
8
9          //Bucle para acumular la suma
10         int acumulador = 0;
11         for (int i = 0; i <= calificaciones.length-1; i++) {
12             acumulador += calificaciones[i];
13         }
14
15         double promedio = (double) acumulador/4;
16         System.out.println("Promedio calculado: " + promedio);
17     }
18 }
19

```

Referencias

Deitel, P. J., & Deitel, H. M. (2017). Java: How to Program (11.a ed.).

Pearson. <https://deitel.com/java-how-to-program-11-e-early-objects-version/>

Joyanes Aguilar, L., & Zahonero Martínez, I. (2014). Programación en Java: Algoritmos, estructura de datos y programación orientada a objetos.

McGraw-Hill. <https://www.mheducation.es/programacion-en-java-6-algoritmos-programacion-orientada-9786071506184-spain>

Sierra, K., & Bates, B. (2005). Head First Java (2.a ed.). O'Reilly Media.

<https://ieeexplore.ieee.org/abstract/document/8883563>

